# CS 255: Computer Architecture and Machine Level Programming

Prof. Martha Barker

Real Numbers and Other Data

# Review: Binary Representation

- Positional Numbers
- Unsigned Binary Integers
- Two’s Complement
- Hexadecimal

# Positional Numbers

The value of a digit depends on its position within the number:

Decimal: $\ldots 10^5\ 10^4\ 10^3\ 10^2\ 10^1\ 10^0$

Binary: $\ldots\ 2^5\ 2^4\ 2^3\ 2^2\ 2^1\ 2^0$

Hexadecimal: $\ldots 16^5\ 16^4\ 16^3\ 16^2\ 16^1\ 16^0$

# Converting to Decimal

Multiply each digit by its positional value and add the results.

$$5683_{10} = 5\times10^3 + 6\times10^2 + 8\times10^1 + 3\times10^0 = 5683_{10}$$

$$1101_2 = 1\times2^3 + 1\times2^2 + 0\times2^1 + 1\times2^0 = 13_{10}$$

$$35d1_{16} = 3\times16^3 + 5\times16^2 + 13\times16^1 + 1\times16^0 = 13777_{10}$$

# Converting from Decimal

Repeatedly divide by the new base and record the remainders, which become the digits from least to most significant.

43/2 = 21 r 1
21/2 = 10 r 1
10/2 = 5 r 0
5/2 = 2 r 1
2/2 = 1 r 0
1/2 = 0 r 1

$43_{10} = 101011_2$

# Representation is Bounded

An n bit number can represent $2^n$ unique values.

*Overflow* occurs when the result of a computation cannot be represented with the number of bits available.

A 4 bit unsigned integer can represent values 0-15:

| | |
|---|---|
| 0110 | 6 |
| + 1100 | 12 |
| 10010 | 18 |

# Two's Complement Signed Integers

The most significant (leftmost) bit has a negative value; all others have positive values (same as unsigned).

$1101_2$ unsigned: 8 + 4 + 1 = 13

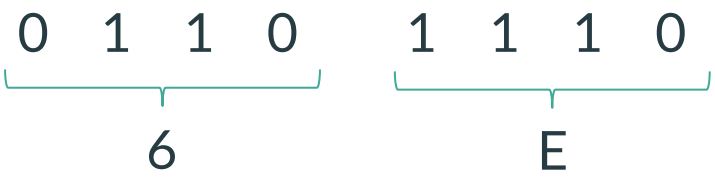
2's comp: -8 + 4 + 1 = -3

Negation:

1. Flip the bits
2. Add one

# Using Hexadecimal as Shorthand

Use hexadecimal as a shorthand for binary:




$01101110_2$ = 0x6E

| Binary | Hex | Decimal |
|---|---|---|
| 0000 | 0 | 0 |
| 0001 | 1 | 1 |
| 0010 | 2 | 2 |
| 0011 | 3 | 3 |
| 0100 | 4 | 4 |
| 0101 | 5 | 5 |
| 0110 | 6 | 6 |
| 0111 | 7 | 7 |
| 1000 | 8 | 8 |
| 1001 | 9 | 9 |
| 1010 | A | 10 |
| 1011 | B | 11 |
| 1100 | C | 12 |
| 1101 | D | 13 |
| 1110 | E | 14 |
| 1111 | F | 15 |

# Real Numbers

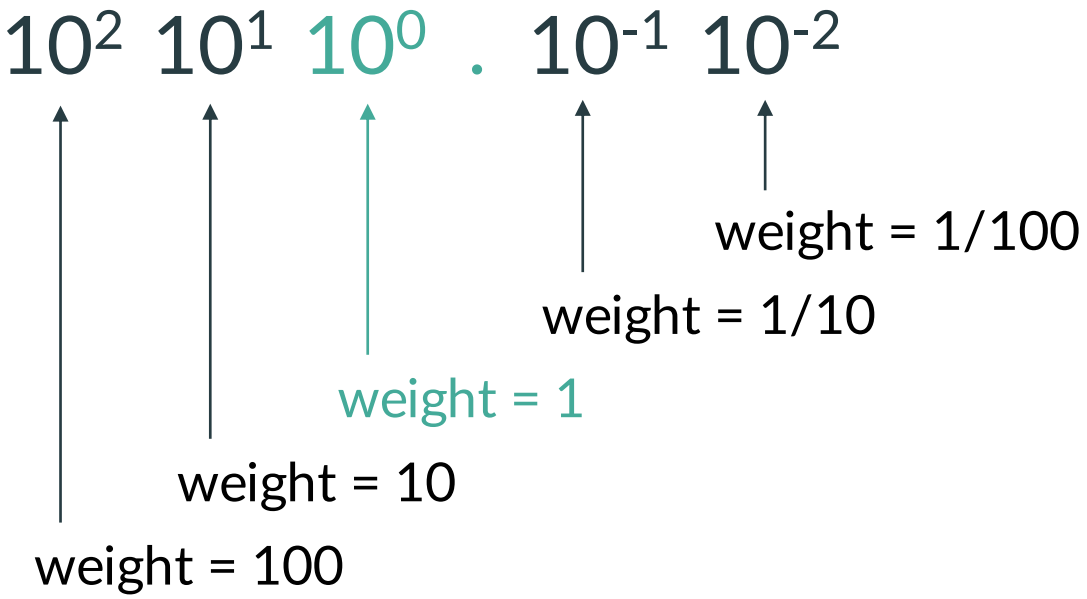
- Fixed Point Representation
- Floating Point Representation
- Rounding Errors

# Not All Numbers are Integers



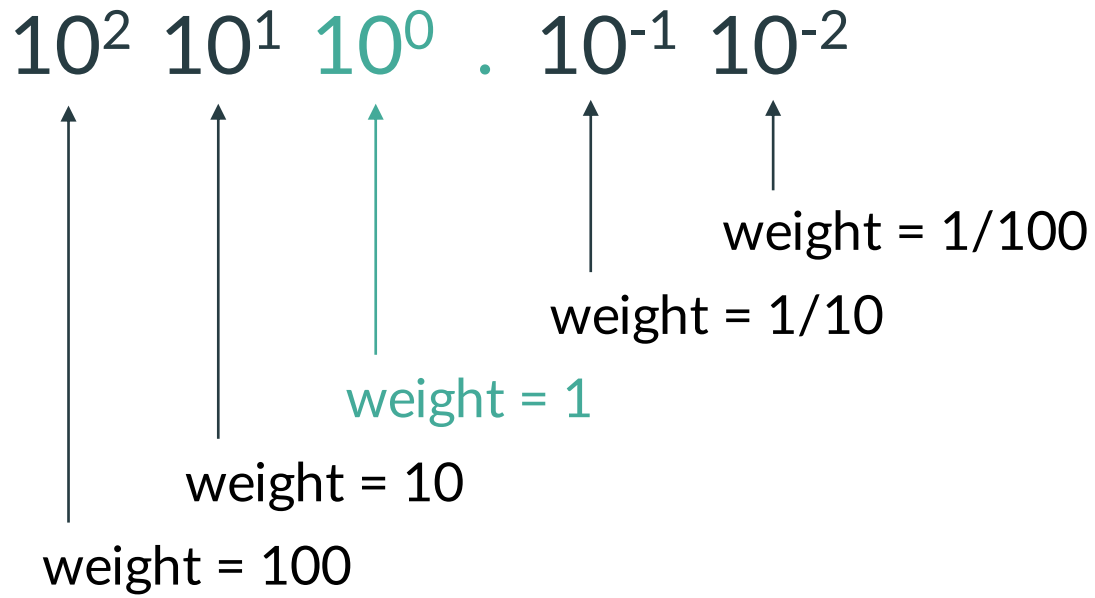
# Fixed Point Representation

The digit to the left of the point has a weight of one.



# Fixed Point Representation

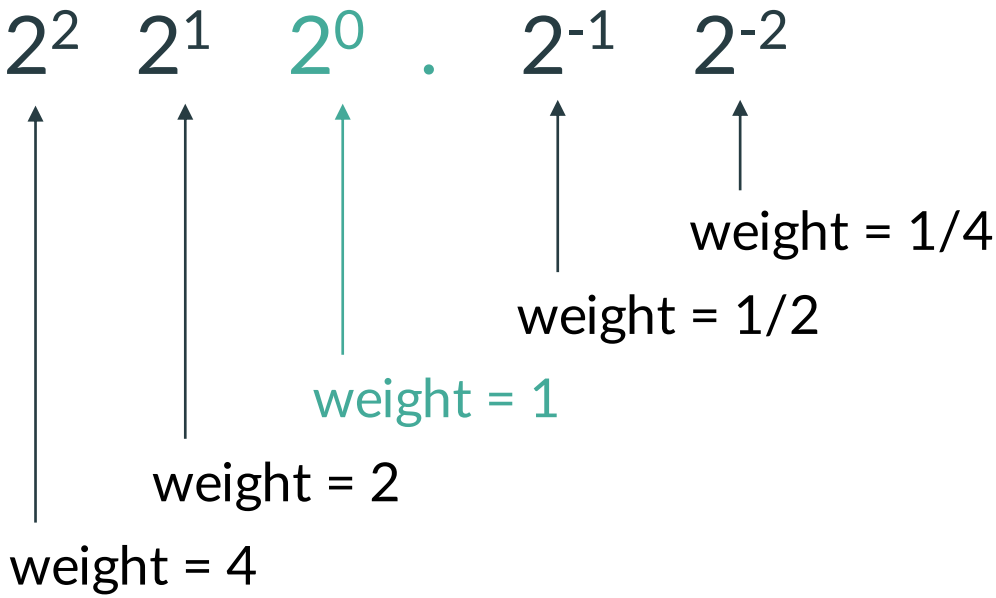
The digit to the left of the point has a weight of one.




$$2.75 = 2\times10^0 + 7\times10^{-1} + 5\times10^{-2}$$

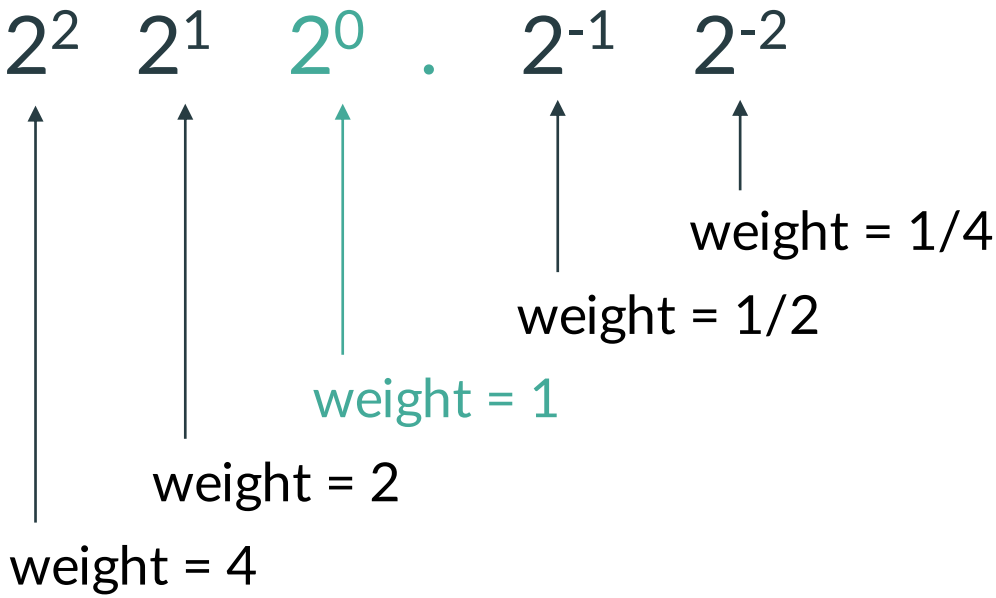
# Fixed Point Representation

The digit to the left of the point has a weight of one.



# Fixed Point Representation

The digit to the left of the point has a weight of one.




$$10.11 = 1\times2^1 + 0\times2^0 + 1\times2^{-1} + 1\times2^{-2}$$

# Converting Decimal to Binary

1. Split the value into the integer part and the fractional part.
2. Convert the integer part by by repeatedly dividing the value by 2.
3. Convert the fractional part by repeatedly multiplying the value by 2.

14.8125 = _ _ _ _ . _ _ _ _

# Converting Decimal to Binary

1. Split the value into the integer part and the fractional part.
2. Convert the integer part by by repeatedly dividing the value by 2.
3. Convert the fractional part by repeatedly multiplying the value by 2.

14.8125 = _ _ _ 0 . _ _ _ _

14/2 = 7 r 0

# Converting Decimal to Binary

1. Split the value into the integer part and the fractional part.
2. Convert the integer part by by repeatedly dividing the value by 2.
3. Convert the fractional part by repeatedly multiplying the value by 2.

14.8125 = _ _ 1 0 . _ _ _ _

14/2 = 7 r 0
7/2 = 3 r 1

# Converting Decimal to Binary

1. Split the value into the integer part and the fractional part.
2. Convert the integer part by by repeatedly dividing the value by 2.
3. Convert the fractional part by repeatedly multiplying the value by 2.

14.8125 = _ 1 1 0 . _ _ _ _

14/2 = 7 r 0
7/2 = 3 r 1
3/2 = 1 r 1

# Converting Decimal to Binary

1. Split the value into the integer part and the fractional part.
2. Convert the integer part by by repeatedly dividing the value by 2.
3. Convert the fractional part by repeatedly multiplying the value by 2.

14.8125 = 1 1 1 0 . _ _ _ _

14/2 = 7 r 0
7/2 = 3 r 1
3/2 = 1 r 1
1/2 = 0 r 1

# Converting Decimal to Binary

1. Split the value into the integer part and the fractional part.
2. Convert the integer part by by repeatedly dividing the value by 2.
3. Convert the fractional part by repeatedly multiplying the value by 2.

14.8125 = 1 1 1 0 . 1 _ _ _

14/2 = 7 r 0
7/2 = 3 r 1
3/2 = 1 r 1
1/2 = 0 r 1

0.8125 × 2 = 1.625

# Converting Decimal to Binary

1. Split the value into the integer part and the fractional part.
2. Convert the integer part by by repeatedly dividing the value by 2.
3. Convert the fractional part by repeatedly multiplying the value by 2.

14.8125 = 1 1 1 0 . 1 1 _ _

14/2 = 7 r 0
7/2 = 3 r 1
3/2 = 1 r 1
1/2 = 0 r 1

0.8125 × 2 = 1.625
0.625 × 2 = 1.25

# Converting Decimal to Binary

1. Split the value into the integer part and the fractional part.
2. Convert the integer part by by repeatedly dividing the value by 2.
3. Convert the fractional part by repeatedly multiplying the value by 2.

14.8125 = 1 1 1 0 . 1 1 0 _

14/2 = 7 r 0
7/2 = 3 r 1
3/2 = 1 r 1
1/2 = 0 r 1

0.8125 × 2 = 1.625
0.625 × 2 = 1.25
0.25 × 2 = 0.5

# Converting Decimal to Binary

1. Split the value into the integer part and the fractional part.
2. Convert the integer part by by repeatedly dividing the value by 2.
3. Convert the fractional part by repeatedly multiplying the value by 2.

14.8125 = 1 1 1 0 . 1 1 0 1

14/2 = 7 r 0
7/2 = 3 r 1
3/2 = 1 r 1
1/2 = 0 r 1

0.8125 × 2 = 1.625
0.625 × 2 = 1.25
0.25 × 2 = 0.5
0.5 × 2 = 1.0

# Fixed Point is Inflexible

The point is in a fixed position, so you must choose between range and precision.

more range, less precision:

_ _ _ _ _ _ _ . _

Unsigned range: 0 to 255.5

Signed range: -128 to 127.5

Step size: 0.5

less range, more precision:

_ . _ _ _ _ _ _ _

Unsigned range: 0 to 1.9921875

Signed range: -1 to 0.9921875

Step size: 0.0078125

# Reminder: Scientific Notation

Scientific notation provides flexible range and precision.

$$834590.316 = 8.34590316 \times 10^{5}$$

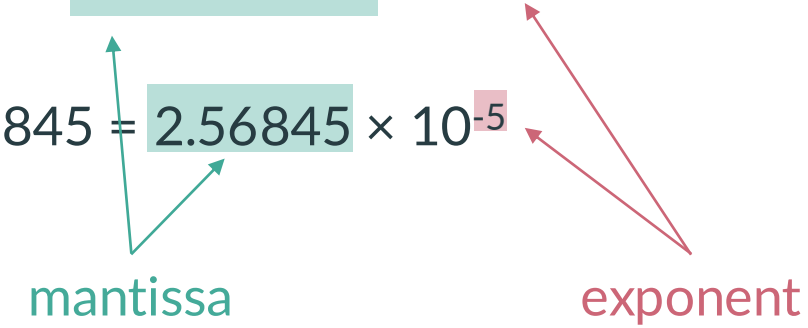
$$0.0000256845 = 2.56845 \times 10^{-5}$$

# Reminder: Scientific Notation

Scientific notation provides flexible range and precision.

$834590.316 = 8.34590316 \times 10^{5}$

$0.0000256845 = 2.56845 \times 10^{-5}$



# Binary Scientific Representation

Binary scientific notation is the same, except with powers of 2.



# Floating Point is Scientific Notation

To encode a floating point number, we need to encode the mantissa and the exponent.

$$100101.1001 = 1.001011001 \times 2^{5} = (1.001011001, 5)$$

$$0.00011010111 = 1.1010111 \times 2^{-4} = (1.1010111, -4)$$

# Floating Point is Scientific Notation

To encode a floating point number, we need to encode the mantissa and the exponent.

$100101.1001 = 1.001011001 \times 2^5 = (1.001011001, 5)$

$0.00011010111 = 1.1010111 \times 2^{-4} = (1.1010111, -4)$

We can (1) normalize so that there is always exactly one non-zero digit to the left of the point and (2) drop the '1.' from our encoding since it is always the same.

# Standardizing Floating Point

IEEE 754 Single Precision Standard




1 sign bit: 0 is positive, 1 is negative
8 exponent bits: exponent represented using excess-127 encoding
23 fraction bits: magnitude of fractional part of mantissa

# Representing the Exponent

Excess-127 encoding shifts the value by 127 to represent both positive and negative values using unsigned bits.

| Decimal Value | Excess-127 Encoding |
|---|---|
| -127 | 0000 0000 |
| -126 | 0000 0001 |
| … | |
| -1 | 0111 1110 |
| 0 | 0111 1111 |
| 1 | 1000 0000 |
| 2 | 1000 0001 |
| … | |
| 127 | 1111 1110 |
| 128 | 1111 1111 |

# IEEE 754 Special Numbers

zero: 0 00000000 00000000000000000000000

infinity: 0 11111111 00000000000000000000000

-infinity: 1 11111111 00000000000000000000000

NaN: 0 11111111 (anything other than all 0)

# Finite Bits Mean Finite Precision

Floating point representation uses a finite number of bits, so it stores the nearest representable value – this results in rounding error.

$$0.1_{10} = 0.000110011001100\ldots_2 \approx 0.100000001490116 12_{10}$$



never compare floats with == or !=

# Exercise: Real Numbers

# Other Data

- Text Data
- Image Data
- Internet Packets
- Video
- ...

# Text Data

ASCII is an encoding standard to represent the English alphabet.

- 1 byte == 1 character
- Each ASCII character has a unique numerical value

Unicode is an encoding standard to represent text in all languages.

- multiple possible encodings (UTF-8, UTF-16, UTF-32)

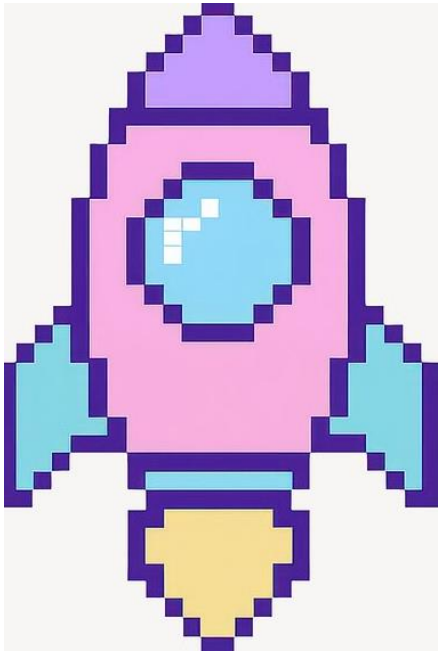
# ASCII TABLE

| Decimal | Hex | Char | Decimal | Hex | Char | Decimal | Hex | Char | Decimal | Hex | Char |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | [NULL] | 32 | 20 | [SPACE] | 64 | 40 | @ | 96 | 60 | ` |
| 1 | 1 | [START OF HEADING] | 33 | 21 | ! | 65 | 41 | A | 97 | 61 | a |
| 2 | 2 | [START OF TEXT] | 34 | 22 | " | 66 | 42 | B | 98 | 62 | b |
| 3 | 3 | [END OF TEXT] | 35 | 23 | # | 67 | 43 | C | 99 | 63 | c |
| 4 | 4 | [END OF TRANSMISSION] | 36 | 24 | $ | 68 | 44 | D | 100 | 64 | d |
| 5 | 5 | [ENQUIRY] | 37 | 25 | % | 69 | 45 | E | 101 | 65 | e |
| 6 | 6 | [ACKNOWLEDGE] | 38 | 26 | & | 70 | 46 | F | 102 | 66 | f |
| 7 | 7 | [BELL] | 39 | 27 | ' | 71 | 47 | G | 103 | 67 | g |
| 8 | 8 | [BACKSPACE] | 40 | 28 | ( | 72 | 48 | H | 104 | 68 | h |
| 9 | 9 | [HORIZONTAL TAB] | 41 | 29 | ) | 73 | 49 | I | 105 | 69 | i |
| 10 | A | [LINE FEED] | 42 | 2A | * | 74 | 4A | J | 106 | 6A | j |
| 11 | B | [VERTICAL TAB] | 43 | 2B | + | 75 | 4B | K | 107 | 6B | k |
| 12 | C | [FORM FEED] | 44 | 2C | , | 76 | 4C | L | 108 | 6C | l |
| 13 | D | [CARRIAGE RETURN] | 45 | 2D | - | 77 | 4D | M | 109 | 6D | m |
| 14 | E | [SHIFT OUT] | 46 | 2E | . | 78 | 4E | N | 110 | 6E | n |
| 15 | F | [SHIFT IN] | 47 | 2F | / | 79 | 4F | O | 111 | 6F | o |
| 16 | 10 | [DATA LINK ESCAPE] | 48 | 30 | 0 | 80 | 50 | P | 112 | 70 | p |
| 17 | 11 | [DEVICE CONTROL 1] | 49 | 31 | 1 | 81 | 51 | Q | 113 | 71 | q |
| 18 | 12 | [DEVICE CONTROL 2] | 50 | 32 | 2 | 82 | 52 | R | 114 | 72 | r |
| 19 | 13 | [DEVICE CONTROL 3] | 51 | 33 | 3 | 83 | 53 | S | 115 | 73 | s |
| 20 | 14 | [DEVICE CONTROL 4] | 52 | 34 | 4 | 84 | 54 | T | 116 | 74 | t |
| 21 | 15 | [NEGATIVE ACKNOWLEDGE] | 53 | 35 | 5 | 85 | 55 | U | 117 | 75 | u |
| 22 | 16 | [SYNCHRONOUS IDLE] | 54 | 36 | 6 | 86 | 56 | V | 118 | 76 | v |
| 23 | 17 | [ENG OF TRANS. BLOCK] | 55 | 37 | 7 | 87 | 57 | W | 119 | 77 | w |
| 24 | 18 | [CANCEL] | 56 | 38 | 8 | 88 | 58 | X | 120 | 78 | x |
| 25 | 19 | [END OF MEDIUM] | 57 | 39 | 9 | 89 | 59 | Y | 121 | 79 | y |
| 26 | 1A | [SUBSTITUTE] | 58 | 3A | : | 90 | 5A | Z | 122 | 7A | z |
| 27 | 1B | [ESCAPE] | 59 | 3B | ; | 91 | 5B | [ | 123 | 7B | { |
| 28 | 1C | [FILE SEPARATOR] | 60 | 3C | < | 92 | 5C | \ | 124 | 7C | \| |
| 29 | 1D | [GROUP SEPARATOR] | 61 | 3D | = | 93 | 5D | ] | 125 | 7D | } |
| 30 | 1E | [RECORD SEPARATOR] | 62 | 3E | > | 94 | 5E | ^ | 126 | 7E | ~ |
| 31 | 1F | [UNIT SEPARATOR] | 63 | 3F | ? | 95 | 5F | _ | 127 | 7F | [DEL] |

# Image Data

An image file is an array of pixels, where each pixel is a (red, green, blue) tuple.

- Each color channel is stored as an integer intensity value
- The three channels determine the pixel's final color when displayed on screen.



# Internet Packets

An IPv6 Packet Header contains information for routing.

- The structured layout of fields specify how the packet is handled
- Each field has a clearly defined role in delivery, priority, and packet identification

**Fixed header format**

| Offsets | Octet | 0 | | | | | | | | 1 | | | | | | | | 2 | | | | | | | | 3 | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Octet | Bit | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 | 31 |
| 0 | 0 | Version | | | | Traffic class | | | | | | | | Flow label | | | | | | | | | | | | | | | | | | | |
| 4 | 32 | Payload length | | | | | | | | | | | | | | | | Next header | | | | | | | | Hop limit | | | | | | | |
| 8 | 64 | Source address | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| 12 | 96 | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| 16 | 128 | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| 20 | 160 | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| 24 | 192 | Destination address | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| 28 | 224 | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| 32 | 256 | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| 36 | 288 | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |

# Streaming Video




Why Snow and Confetti Ruin YouTube Video Quality